

Vyapar AI — Technical & Business Architecture Summary

Generated from a full read of the `main` branch.

1. Tech Stack & Architecture

Backend framework

- **Python + FastAPI** (ASGI), served by **Uvicorn**. App, lifespan, and all HTTP routes are in `main.py` (entry point: `python main.py`, default port `10000`).
- **Server-rendered UI** using **Jinja2** templates + **Tailwind CSS (CDN)** for the owner dashboard and the public chat widget (`templates/` , `dashboard.py` , `web_widget.py`). There is **no separate JS frontend / SPA**.
- Dashboard **auth** is custom and dependency-light: **PBKDF2-HMAC-SHA256** password hashing + an **HMAC-signed session cookie** (`auth.py`).

Database

- **SQLAlchemy 2.0 ORM** (`memory_db.py`) with a `get_session()` context manager.
- **SQLite** for local development (`DATABASE_URL=sqlite:///./data/vyapar.db`).
- **PostgreSQL** for production via `DATABASE_URL` using the **psycopg 3** driver. `postgres://` / `postgresql://` are auto-rewritten to `postgresql+psycopg://` .
- Schema created at startup via `init_db()` (`Base.metadata.create_all`) — **no Alembic/migrations**.
- **Tables:** `user_memory` , `conversation_context` , `chat_turn` , `conversation_state` , `leads` , `company_profile` , `owner_user` , `invoice` .

AI / LLM Integration

- **Primary: OpenAI Responses API** (`openai_engine.py`) via `AsyncOpenAI().responses.create(...)` . Model configurable through `OPENAI_MODEL` (default `gpt-5.4-mini`); tunables: `OPENAI_TEMPERATURE` , `OPENAI_MAX_OUTPUT_TOKENS` , `OPENAI_TIMEOUT_SECONDS` , `OPENAI_STORE` . Includes retry/backoff.
- **Optional fallback: Google Gemini** (`google-genai`) when `ENABLE_GEMINI_FALLBACK=true` (`GEMINI_MODEL` , default `gemini-2.5-flash`).
- **Prompt locations:** `prompts.py` (`BASE_SYSTEM_PROMPT` , `compose_system_prompt()`), `business_settings.py` (tenant profile block), `intent_engine.py` (`INTENT_HINTS`), `ai_employee_engine.py` (`_build_user_prompt` , `_turn_router_prompt` , orchestration in `generate_employee_reply`).
- The AI "brain" `generate_employee_reply(user_id, text, company_id)` is **channel-agnostic**.

Hosting & Deployment

- **Render**, configured in `render.yaml` : `runtime: python` , `plan: starter` , `buildCommand: pip install -r requirements.txt` , `startCommand: python main.py` , `healthCheckPath: /healthz` , `autoDeploy: true` .
- Baked env vars: `TELEGRAM_MODE=webhook` , `PORT=10000` , `OPENAI_MODEL=gpt-5.4-mini` , `OPENAI_STORE=false` . Full env setup in `README_DEPLOY.md` .
- **GitHub workflow: none** — no `.github/` directory, so no GitHub Actions CI/CD; deploys rely on Render `autoDeploy` .
- Health/status endpoints: `GET /healthz` (DB ping) and `GET /` (service + active tenant).

2. Integration & Channels

Implemented channels

Channel	Status	Code	Inbound	Outbound
Telegram	Implemented	<code>main.py</code>	<code>POST /telegram/webhook</code> + polling	Bot API <code>sendMessage</code>
Facebook Messenger	Implemented	<code>facebook_messenger.py</code>	<code>GET/POST /facebook/webhook</code> (verify token + <code>X-Hub-Signature-256</code>)	Graph API <code>me/messages</code>
Web Chat Widget	Implemented	<code>web_widget.py</code>	<code>POST /widget/{widget_key}/message</code>	JSON reply to embedded iframe
WhatsApp / Instagram / Viber	Not implemented	—	"Coming soon" in <code>templates/channels.html</code>	—

- Telegram & Facebook tokens are **global env vars** (one bot/page per server). The **Web Widget is the only truly per-tenant-routed channel**, via a public `widget_key` resolved by `company_manager.get_company_by_widget_key()`. This widget pattern is the template for making other channels per-tenant.

Data flow: customer message to dashboard

- Channel adapter receives a message (`web_widget.widget_message` , `main.handle_update` , `facebook_messenger.handle_webhook_payload`).
- It calls `generate_employee_reply(user_id, text, company_id)` . User IDs are channel-namespaced (`web:{key}:{session}` , `fb:{sender}`).
- Per turn the engine: writes user + assistant messages to `chat_turn` ; syncs `conversation_state` (language, stage, escalation, turn count); extracts/validates personal facts into `user_memory` ; runs `extract_lead_bundle()` and, if `should_process_lead()` , calls `upsert_lead()` into `leads` .
- The **dashboard** reads it: Conversations inbox lists customers from `conversation_state` (tenant-scoped) and shows transcripts from `chat_turn` ; Leads reads `leads` by `company_id` .
- Note:* `chat_turn` has **no** `company_id` (keyed by `user_id`); tenant conversation scoping is derived via `conversation_state` .

3. Business Logic & Lead Funnel

Lead schema

`leads` (`memory_db.Lead`): `stage` , `lead_score` , `customer_name` , `location` , `budget` , `requested_speed` , `phone` , `contact_method` , `contact_value` , `urgency` , `buying_intent` , `coverage_check_needed` , `coverage_area` , `coverage_status` , `matched_product` , `alternative_product` , `last_discussed_product/question/reply` , `signals_json` , timestamps.

Lead scoring (`lead_extractor.compute_lead_score` , 0–100)

- Buying intent **+25**; requested speed **+15**; location **+15**; budget **+10**.
- Urgency: high **+10**, medium **+5**.
- Valid phone **+20**; WhatsApp **+18**; Telegram contact **+5**; name **+5**; coverage-check needed **+5**.

Stage triggers (`lead_extractor.derive_stage`)

- new** — no buying intent and no signals (speed/location/budget/urgency).
- interested** — `buying_intent` OR `lead_score >= 40` OR has location/speed/budget.
- qualified** — (valid phone OR WhatsApp) **AND** (location OR requested speed). Score alone is **not** sufficient.
- hot** — qualified criteria met **AND** (urgency `high` , OR urgency `medium` with `lead_score >= 75`).

- **Monotonic:** `lead_manager._max_stage()` never downgrades a lead; `lead_score` kept as `max(existing, new)`.
- A parallel "sales mode" gate (`ai_employee_engine._resolve_sales_mode`) flips selling behavior when stage in {interested, qualified, hot}, `score >= 40` , buying intent, or coverage check needed.
- **Hot-lead alerting:** `admin_notifier.maybe_notify_admin()` fires on thresholds when `ADMIN_ALERT_ENABLED=true` — currently a **log-only placeholder** (`ADMIN_ALERT_PLACEHOLDER`).

Products

- Stored **inside the tenant's company profile JSON** (`company_profile.data_json` → `products`), not a dedicated table; normalized for prompts by `company_manager.get_company_products()` . Managed via the dashboard Products CRUD page.

Billing / Outstanding

- `invoice` table (`memory_db.Invoice`): `company_id` , `customer_name/phone` , `description` , `amount` , `currency` , `status` (unpaid/paid), `due_date` , `paid_at` . "Overdue" computed (unpaid + past due).
- Managed in `dashboard.py` billing routes: create, mark-paid, delete, CSV export, summary (total billed / paid / outstanding / overdue). Outstanding = sum of unpaid amounts.
- **Third-party billing/payment API placeholders: none.** Billing is **manual** ("Mark paid"); **no payment-gateway integration** (no eSewa/Khalti/Stripe).

4. Core System Prompt / Instructions

Assembled by `compose_system_prompt()` (`prompts.py`), layering: base persona, language lock, business profile, intent hint, turn-router rule, session state, memory, lead context, knowledge base, product block.

Base persona (`BASE_SYSTEM_PROMPT`)

You are Vyapar AI Employee, a production business assistant for Nepali business use.

Behavior rules: - Reply in the same language as the user unless they explicitly ask for another language. - Be concise, helpful, practical, and professional. - Prefer verified memory and provided business knowledge over guessing. - If a fact such as price, billing status, order status, or availability is not provided, say you do not have that confirmed information yet. - Never invent personal memory about the user. - If the user asks about their own saved details and those details are present in the prompt, use them directly. - If the user shares new personal facts, it is acceptable to acknowledge them naturally. - Avoid long disclaimers. Focus on action. - Telegram messages should be reasonably short and easy to read.

Sales persona overlay (`SALES_EMPLOYEE_HINT`)

Act as a sales employee, not a FAQ bot; answer the latest message first; acknowledge the specific need; move toward qualification/next step; if exact product unavailable, propose only the suggested alternative; if coverage pending, don't promise installation; ask for phone/WhatsApp when intent is strong; don't repeat a recent pitch.

Anti-repeat overlay (`ANTI_REPEAT_HINT`)

Read recent turns and the last sales reply; never copy/paraphrase the previous pitch; address any new concern (price, discount, competitor, hesitation, escalation) first.

Coverage overlay (`COVERAGE_SALES_HINT`)

When an ISP service area isn't confirmed: say the team will verify, don't promise a date, keep qualifying.

Per-intent + per-turn guidance

`intent_engine.INTENT_HINTS` provides distinct instructions per intent (buying_intent, pricing, coverage_inquiry, support, billing, complaint, greeting, general_knowledge, identity), and `ai_employee_engine._turn_router_prompt()` adds per-turn rules (greeting, memory_query, memory_write, escalation, objection, etc.).

Key Architectural Observations

- **Multi-tenancy** is keyed on `company_id` ; profiles are DB-backed (`company_profile`) and seeded once from `company_profiles.json` . Dashboard, leads, billing, and web widget are all tenant-scoped.
- **Industry coupling:** the NLU layer (`intent_engine.py` , `lead_extractor.py`) and some prompt overlays are **ISP-specific** (Mbps, "internet jodne", coverage checks) and **romanized-Nepali tuned**. Generalizing per industry is the main work to make the engine fully horizontal.
- **Channel tokens** for Telegram/Facebook are still **global env vars**; only the Web Widget routes per-tenant today.
- **No automated tests, no linter config, no CI** in the repo.
- **Payments and delivery** are not yet implemented (billing is manual invoice tracking).